Testing Document — Personalized Weather Station

Purpose of Testing

The purpose of testing was to verify that the Personalized Weather Station design and supporting software successfully demonstrated the intended environmental monitoring functionality.

Testing focused on:

- Environmental sensor operation
- USB serial communication
- Data processing
- Software visualization
- Notification functionality
- PCB manufacturability and validation

The testing process also helped identify hardware design issues, communication problems, and missing support circuitry during development.

Test 1 — Power Verification

Objective

Verify that the PCB powers correctly and distributes voltage properly.

Procedure

1. Connect the dedicated USB-C power connector.
2. Apply power to the board.
3. Measure voltage rails using a multimeter.
4. Verify voltage regulator output.
5. Observe status LED behavior.

Expected Results

- Board powers on successfully.
- Voltage regulator outputs stable 3.3V.
- No excessive heat or short circuits are observed.
- Status LED functions correctly.

Results

Power distribution functioned as expected during testing. Voltage regulation was added specifically to provide stable 3.3V operation for the MCU and BME280 sensor.

Test 2 — Environmental Sensor Validation

Objective

Verify that the BME280 sensor correctly measures environmental conditions.

Procedure

1. Connect the BME280 sensor to the PCB.
2. Flash firmware to the ATmega328P-A.
3. Monitor serial output from the MCU.
4. Observe temperature, humidity, and pressure readings.
5. Compare changing values with surrounding environmental conditions.

Expected Results

- Sensor initializes correctly.
- Temperature readings update continuously.
- Humidity readings update continuously.
- Pressure readings update continuously.
- Values respond to environmental changes.

Results

The BME280 successfully provided environmental readings during testing and demonstration. Earlier development stages experienced issues due to incorrect sensor pin labeling, but replacement sensors and corrected wiring resolved the issue.

Test 3 — USB Serial Communication Test

Objective

Verify communication between the PCB and external computer software.

Procedure

1. Connect the dedicated USB-C communication connector to the computer.
2. Verify the COM port appears on the computer.
3. Launch the Python application.
4. Open the serial connection.
5. Observe incoming environmental data.

Expected Results

- Computer detects the device.
- Serial communication initializes successfully.
- Environmental data transfers continuously.
- No communication interruptions occur during operation.

Results

USB serial communication successfully transferred environmental data from the PCB to the Python application during demonstration testing.

Test 4 — Python Application Validation

Objective

Verify that the Python application correctly processes and visualizes environmental data.

Procedure

1. Launch the Python monitoring application.
2. Connect the serial port.

3. Observe incoming sensor readings.
4. Verify graph updates.
5. Verify prediction and weather classification logic.

Expected Results

- Python application launches successfully.
- Environmental values display correctly.
- Graphs update continuously.
- Prediction logic processes incoming data.
- Weather classifications display correctly.

Results

The Python application successfully received and displayed environmental readings during demonstration testing. The software was able to graph readings, estimate short-term trends, and classify weather conditions.

Test 5 — ntfy Notification Validation

Objective

Verify that environmental notifications can be sent to a phone using ntfy.

Procedure

1. Run the Python application.
2. Trigger notification logic.
3. Observe notification behavior.
4. Verify phone receives ntfy message.

Expected Results

- ntfy messages send successfully.
- Notifications appear on the phone.
- Notification cooldown logic prevents excessive alerts.

Results

The ntfy notification system successfully demonstrated phone alert capability during testing.

Test 6 — PCB Validation & Design Verification

Objective

Verify that the PCB design is manufacturable and electrically valid.

Procedure

1. Run ERC verification in KiCad.
2. Run DRC verification in KiCad.
3. Review routing and spacing.
4. Verify footprints and component placement.
5. Generate fabrication files.

Expected Results

- ERC completes successfully.
- DRC completes successfully.
- PCB routing satisfies design rules.
- Gerber and drill files generate correctly.

Results

The PCB design was successfully prepared for manufacturing. Gerber files, drill files, BOM documentation, and PCB ordering instructions were generated and organized for fabrication.

Major Issues Encountered During Testing

BME280 Sensor Damage

Some BME280 modules were damaged during early testing because the pins were incorrectly labeled or misidentified.

Resolution

- Replacement sensors were obtained.
- Wiring was corrected.
- Sensor operation was revalidated.

Missing Support Components

Early PCB review identified missing decoupling and support components.

Resolution

- Additional capacitors were added.
- Support circuitry was updated.
- PCB design was reviewed again.

PCB Routing Constraints

Routing congestion occurred during PCB layout development.

Resolution

- Routing strategy was revised.
- 4-layer PCB concepts were explored.
- Via assignment and inner-layer routing techniques were tested.

Overall Testing Conclusion

Testing successfully demonstrated:

- Environmental sensing functionality
- USB serial communication
- Python-based monitoring and graphing

- ntfy notification capability
- PCB manufacturability
- Embedded system integration

The project evolved significantly throughout testing and development. Early proof-of-concept work using a Raspberry Pi and DHT22 sensor eventually transitioned into a more realistic custom PCB-based embedded system using the ATmega328P-A and BME280.

The final system demonstrated successful environmental monitoring, software integration, manufacturing preparation, and practical embedded system development workflows.